Remco Bloemen
Math & Engineering
https://2π.com

Discrete Convolutions

19 September 2023 (updated 24 July 2024)

Given two sequence a_i and b_i their *discrete convolution*, denoted $a * b$, is a sequence c_i such that

$$c_i = \sum_j a_j \cdot b_{i-j}$$

In practice the sequences are finite and variations of convolution depend on how the boundary is handled. In a linear convolution the sequences are zero-extended where missing. In a cyclic convolution a, b and c have the same length n and indices are taken modulo n, i.e. they are periodic. If b is periodic except for the sign changing every period it is a negacyclic convolution. Finally, correlation, denoted $c = a \star b$, is the same but with b reversed, i.e. $c_i = \sum_j a_j \cdot b_{i+j}$. In linear algebra, linear and cyclic convolution are equivalent to multiplying by Toeplitz or circulant matrix respectively. In digital signal processing, linear convolutions are known as finite impulse response filters. In machine learning, convolutional neural networks make use of linear convolutions. They also show up in many FFT and NTT algorithms.

In algebra, discrete convolutions appears as polynomial multiplication. Given $A, B, C \in \mathbb{F}[X]$ such that $C(X) = A(X) \cdot B(X)$. Write all three coefficient form like $A(x) = a_0 + a_1 \cdot X + a_2 \cdot X^2 \cdots$ then $c = a * b$. This is the core of the large integer multiplication problem. To see this consider that a number like 123 equals the polynomial $3 + 2 \cdot X + 1 \cdot X^2$ evaluated at $X = 10$. To multiply two large numbers one multiplies the polynomials and then does carry propagation. The resulting inner multiplications will have argument of size equal to the chosen base. When this base is itself large the method can be employed recursively.

Using polynomials (quotient) rings we can concisely define linear (LCV), cyclic (CCV) and negacyclic (NCV) convolution as multiplication in $\mathbb{F}[X]$, $\mathbb{F}[X]/(X^n - 1)$ and $\mathbb{F}[X]/(X^n + 1)$ respectively:

LCV	$C(X) = A(X) \cdot B(X)$
CCV	$C(X) = A(X) \cdot B(X) \mod X^n - 1$
NCV	$C(X) = A(X) \cdot B(X) \mod X^n + 1$

A direct computation of $n \times m$ linear convolution takes $(n \cdot m - n - m + 1) \cdot \mathsf{A} + n \cdot m \cdot \mathsf{M}$. For a (nega) cyclic convolution this is $n \cdot (n-1) \cdot \mathsf{A} + n^2 \cdot \mathsf{M}$.

Toom-Cook methods

Polynomials can be represented in many bases. The most common is the monomial basis, i.e. coefficient form. But they can also be represented by their evaluations on an (arbitrary) set of points $\{x_i\}$, called a Lagrange basis. The Lagrange bases have the unique advantage that polynomial multiplication becomes a simple element-wise product:

$$C(x_i) = A(x_i) \cdot B(x_i)$$

Computing this takes only n multiplication operations, where n is the number of terms in C. To use it for convolutions we also need to convert between monomial basis and lagrange basis (i.e. coefficients and evaluations). This is a basis transformation in the linear algebra sense, so we can represent it as a so-called Vandermonde matrix V:

$$V = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots \\ 1 & x_1 & x_1^2 & \cdots \\ 1 & x_2 & x_2^2 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

We can now write the Toom-Cook convolution algorithm using matrix-vector products $\cdot$ and element-wise products $\circ$:

$$c = V^{-1} \cdot ((V \cdot a) \circ (V \cdot b))$$

The evaluation points $\{x_i\}$ are arbitrary, so which points do we pick to make computations as easy as possible? Some great first candidates are $\{0, 1, -1\}$, which respectively result in the first coefficient, the sum of the coefficients and the alternating sum of the coefficients; no multiplications required.

Point at infinity

Convolutions are symmetrical under reversing all the sequences. In the polynomial version this means reversing the order of the coefficients, or equivalently $P'(X) = P(X^{-1}) \cdot X^{n-1}$. Each previous evaluation point has a corresponding point $x' = x^{-1}$ in the reverse polynomials, except for 0 which has no inverse. Instead, we get a new evaluation point $x' = 0$ that has no pre-image. Evaluating at the new point $P'(0)$ gives p_{n-1} much like $P(0) = p_0$. These are both trivial to compute, so we like to add both to our evaluation set. Somewhat informally we refer to the $x' = 0$ point as the 'point at infinity' $x = \infty$. Fortunately there is an easy way to add it to a Vandermonde matrix: coefficient reversal is identical to reversing a row in the matrix, so the row corresponding to $x = \infty$ is the row corresponding $x = 0$ reversed, i.e. $\begin{bmatrix} 0 & 0 & \cdots & 1 \end{bmatrix}$.

Karatsuba

Consider a 2×2 linear convolution. The result is $c = [a_0 \cdot b_0,\ a_0 \cdot b_1 + a_1 \cdot b_0,\ a_1 \cdot b_1]$ which takes four multiplications when evaluated directly. If we pick the basis $\{0, 1, \infty\}$ we end up with the following algorithm for a 2×2 linear convolution:

$$V = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \qquad V^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix} \cdot \left(\left(\begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} \right) \circ \left(\begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} b_0 \\ b_1 \end{bmatrix} \right) \right)$$

Note that when we trim the V matrix we should keep the 1 in the last column for the point at infinity. Writing out all the multiplications we get

$$\begin{aligned} y_0 &= a_0 \cdot b_0 & c_0 &= y_0 \\ y_1 &= (a_0 + a_1) \cdot (b_0 + b_1) & c_1 &= y_1 - y_0 - y_2 \\ y_2 &= a_1 \cdot b_1 & c_2 &= y_2 \end{aligned}$$

We now have three multiplications instead of four, though we went from one addition to four. This specific instance of Toom-Cook is known as the Karatsuba algorithm. It is the core of the first sub-quadratic method for multiplication and led to the development of Toom-Cook and other methods.

Toom-3

For a 3×3 linear convolution we can use the points $\{0, 1, -1, -2, \infty\}$, these are all the easy ones with the addition of -2. The choice of -2 leads to further optimizations due to Bod07.

$$V = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 \\ 1 & -2 & 4 & -8 & 16 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \qquad V^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{3} & -1 & \frac{1}{6} & -2 \\ -1 & \frac{1}{2} & \frac{1}{2} & 0 & -1 \\ \frac{1}{2} & -2 & -\frac{1}{2} & -\frac{1}{6} & 2 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix} = V^{-1} \cdot \left(\left(\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & -2 & 4 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} \right) \circ \left(\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & -2 & 4 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} \right) \right)$$

Both V and V^{-1} allow for more efficient evaluation than direct. Take $t = a_0 + a_2$ then $a' = V \cdot a$ can be computed as

$$\begin{aligned} a'_0 &= a_0 \\ a'_1 &= t + a_1 \\ a'_2 &= t - a_1 \\ a'_3 &= 2 \cdot (a'_2 + a_2) - a_0 \\ a'_4 &= a_2 \end{aligned}$$

using add for doubling, this gives $5 \cdot \mathsf{A}$ instead of $7 \cdot \mathsf{A} + \mathsf{M^c}$. Similarly $V^{-1} \cdot y$ can be computed in $9 \cdot \mathsf{A} + 3 \cdot \mathsf{M^c}$ instead of $14 \cdot \mathsf{A} + 9 \cdot \mathsf{M^c}$ using

$$\begin{aligned} t_1 &= \frac{1}{2} \cdot (y_1 - y_2) & c_0 &= y_0 \\ t_2 &= y_2 - y_0 & c_2 &= t_2 + t_1 - y_4 \\ t_3 &= \frac{1}{3} \cdot (y_3 - y_1) & c_3 &= \frac{1}{2} \cdot (t_2 + t_3) + 2 \cdot y_4 \\ & & c_1 &= t_1 - c_3 \\ & & c_4 &= y_4 \end{aligned}$$

This brings the total to $19 \cdot \mathsf{A} + 3 \cdot \mathsf{M^c} + 5 \cdot \mathsf{M}$ compared to $4 \cdot \mathsf{A} + 9 \cdot \mathsf{M}$ for direct convolution.

Question. Are there algorithms to find the optimal evaluation for a given matrix?

For larger sizes GMP uses the point set $\{0, \infty, +1, -1, \pm 2^i, \pm 2^{-i}\}$. This gives some implementation optimization opportunities, but doing the evaluation/interpolation step efficiently becomes increasingly difficult for larger sets. Without evaluation tricks this would result in $() \cdot \mathsf{A} + () \cdot \mathsf{M^c} + (2 \cdot n - 1) \cdot \mathsf{M}$ for an $n \times n$ linear convolution.

$\mathsf{M^c}$ $(2 \cdot n - 5) \cdot (n - 1)$ for V. $(2 \cdot n - 3) \cdot (2 \cdot n - 1)$ for V^{-1}.

Convolution theorem

If we pick a n-th order root of unity ω_n and uses its powers as $\{\omega_n^0, \omega_n^1, \ldots, \omega_n^{n-1}\}$ as evaluation points, then our Vandermonde matrix V also becomes a DFT Matrix and the evaluation/interpolation steps become Fourier transforms NTT_n. This result is known as the convolution theorem.

$$c = \mathsf{NTT}_n^{-1}(\mathsf{NTT}_n(a) \circ \mathsf{NTT}_n(b))$$

This was first used for very large integer multiplication with the carefully optimized Schönhage–Strassen algorithm. This algorithm was further improved leading to the Harvey-van der Hoeven algorithm which runs in $\mathcal{O}(n \cdot \log n)$.

Bilinear algorithms

We can generalize the algorithms considered so far as special cases of *bilinear algorithms* of the form

$$c = C \cdot ((A \cdot a) \circ (B \cdot b))$$

where $A \in \mathbb{F}^{r \times n_a}$, $B \in \mathbb{F}^{r \times n_b}$ and $B \in \mathbb{F}^{n_c \times r}$ for some r called the *bilinear rank*. To motivate this generalization consider the following bilinear algorithm for 3×3 linear convolution due to Bla10 (§5.4):

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & 0 & 1 & 0 & 0 \\ -1 & 1 & -1 & 0 & 1 & 0 \\ 0 & -1 & -1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \cdot \left(\left(\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} \right) \circ \left(\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} \right) \right)$$

It has one rank higher than Toom-3, but at $13 \cdot \mathsf{A} + 6 \cdot \mathsf{M}$ it has fewer additions and no multiplications by constants. It can also not be formulated as evaluation on some set of points. (**Question.** Can it be formulated as a Winograd factorization?)

Usually A, B and C are sparse by construction. The number of operations required for such matrices are at most

$$(m - n) \cdot \mathsf{A} + k \cdot \mathsf{M}^{\mathsf{c}}$$

where n is the number of rows, m is the number of non-zero elements and k the number of elements not equal to $0, 1$ or -1. Note that this is an upper bound, as in the example of

The Matrix Exchange Theorem

In Toom-Cook we have nice A and B matrices, but C becomes unwieldy. Fortunately, there is a trick to swap C with A or B. This is useful as we can often precompute $A \cdot a$ or $B \cdot b$. Without loss of generality we'll assume swapping C and B.

Start with the observation that $x \circ y = \mathrm{diag}(x) \cdot y$ where $\mathrm{diag} : \mathbb{F}^n \to \mathbb{F}^{n \times n}$ is the vector-to-diagonal-matrix operator. Furthermore, given matrices A, B and diagonal matrix D we have $A \cdot D \cdot B = (B \cdot \mathrm{E})^\mathsf{T} \cdot D \cdot (\mathrm{E} \cdot A)^\mathsf{T}$ where E is the exchange matrix (see Bla10 5.6.1). (**To do**. This requires further restrictions such that the matrix is persymmetric) Note that for any matrix M, $\mathrm{E} \cdot M$ is just M with the rows in reverse order and $M \cdot \mathrm{E}$ has the columns in reversed. Applying this to our bilinear algorithm results in

$$\begin{aligned} c &= C \cdot ((A \cdot a) \circ (B \cdot b)) \\ &= C \cdot \mathrm{diag}(A \cdot a) \cdot B \cdot b \\ &= (B \cdot \mathrm{E})^\mathsf{T} \cdot \mathrm{diag}(A \cdot a) \cdot (\mathrm{E} \cdot C)^\mathsf{T} \cdot b \\ &= (B \cdot \mathrm{E})^\mathsf{T} \cdot \Big((A \cdot a) \circ \Big((\mathrm{E} \cdot C)^\mathsf{T} \cdot b \Big) \Big) \end{aligned}$$

thus we have a new bilinear algorithm where B and C are swapped, transposed and have their rows/columns reversed.

Question. Does this apply to all kinds of convolutions?

Correlations

Given an algorithm (A, B, C) for linear convolution, an algorithm for correlation is constructed by (A, C, B). This is the Matrix Interchange theorem in Bla10.

Two-dimensional convolutions

Given a $n \times m$ convolution (A_1, B_1, C_1) and a $k \times l$ convolution (A_2, B_2, C_2) we can construct a two-dimensional $(n, k) \times (m, l)$ convolution as

$$(A_1 \otimes A_2, B_1 \otimes B_2, C_1 \otimes C_2)$$

where the inputs are vectorized with inner dimensions k, l, i.e. $(i, j) \mapsto i \cdot k + j$ and $i \cdot l + j$ respectively. This result holds for linear, cyclic and negacyclic convolutions.

Question. Does this hold for general polynomial product mod m? Does this hold in general for bilinear maps?

Overlap-add

Furthermore, we can use this to construct a $(n \cdot k) \times (m \cdot l)$ linear convolution by recombining the outputs.

$$P(X) = P_0(X) + P_1(X) \cdot X^2$$
$$Q(X) = Q_0(X) + Q_1(X) \cdot X^2$$

$$P(X) \cdot Q(X) =$$

$$Q$$

Agrawal-Cooley

Modular reduction

Given a polynomial $P(X) = \sum_i p_i \cdot X^i$ and a monic modulus $M(X) = \sum_i m_i \cdot X^i$ we want to compute

$$Q(X) = P(X) \mod M(X)$$

$$\begin{bmatrix} q_0 \\ q_1 \\ \vdots \\ q_{m-1} \end{bmatrix} \left[\begin{array}{c|cccc} \mathrm{I}_m & m_0 & m_{m-2} & m_{m-3} & \cdots \\ & m_1 & m_0 & m_{m-2} & \cdots \\ & m_2 & m_1 & m_0 & \\ & \vdots & \vdots & \vdots & \ddots \\ & m_{m-2} & m_{m-3} & m_{m-4} & \cdots \end{array}\right] \cdot \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_{n-1} \end{bmatrix}$$

$$\left[\begin{array}{c|c} \mathrm{I}_m & \mathrm{Toeplitz}\left(\begin{bmatrix} m_0 & m_1 & \cdots & m_{m-2} \end{bmatrix}\right) \end{array}\right]$$

Where the Toeplitz matrix is extended with however many columns necessary.

In particular this can be used to turn linear convolution algorithms into (nega)cyclic by reducing modulo $X^n \pm 1$.

Winograd Convolution

A generalization of Toom-Cook can be made be realizing that evaluating a polynomial at a point $P(x_i)$ is the same as reducing it modulo $X - x_i$. We are then solving the system of equations

$$C(X) = A(X) \cdot B(X)[X - x_i]$$

and then reconstructing $C(X)$ from the residues mod $X - x_i$. This is a special case of the Chinese Remainder Theorem (CRT). Generalizing we pick a set of co-prime moduli polynomials $\{M_i(X)\}$, their product $M(X) = \prod_i M_i(X)$ and a CRT basis $\{E_i(X)\}$ (see appendix), compute

$$C_i(X) = (A(X) \cdot B(X) \mod M_i(X))$$
$$C(X) = \sum_i C_i(X) \cdot E_i(X)$$

If $M(X) = X^n - 1$ we have cyclic convolution, and with $M(X) = X^n + 1$ negacyclic.

If $R_i(X)$ is a multiple of $M_i(X)$, then $E_i(X)$ is a multiple of $M(X)$, which reflects the fact that CRT is only unique up to $\mod M(X)$.

Question. Another generalization of point evaluations is to include derivatives at those points. How does this generalization combine with CRT?

Question. Can all bilinear algorithms for convolutions be interpreted as Winograd convolutions? Does this extend to non-convolution bilinear algorithms?

Recursion

This method allows for recursion. For example to compute a $2 \cdot n$ sized cyclic convolution we can pick a basis $M(X) = (X^n + 1) \cdot (X^n - 1)$ and get

$$C(X) = A(X) \cdot B(X)[X^n - 1]$$
$$C(X) = A(X) \cdot B(X)[X^n + 1]$$

$$E_0(X) = R_0(X) \cdot (X^n + 1) \qquad R_0(X) = (X^n + 1)^{-1}[X^n - 1]$$
$$E_1(X) = R_1(X) \cdot (X^n - 1) \qquad R_1(X) = (X^n - 1)^{-1}[X^n + 1]$$

$$X^n + 1 \mod (X^n - 1) = 2$$

$$X^n - 1 \mod (X^n + 1) = -2$$

So $R_0(X) = \frac{1}{2}$ and $R_1(X) = -\frac{1}{2}$.

$$E_0(X) = \frac{1}{2} \cdot (X^n + 1) \qquad E_1(X) = -\frac{1}{2} \cdot (X^n - 1)$$

Re-introducing the point at infinity

We can add a $M_i(X) = X - \infty$.

$$X^{n_c - 1} \cdot C(X^{-1}) = \left(X^{n_a - 1} \cdot A(X^{-1})\right) \cdot \left(X^{n_b - 1} \cdot B(X^{-1})\right)[X - 0]$$

Larger convolutions

Nested Convolutions

Agarwal-Cooley

Lower bounds

Linear convolution can not be computed with fewer than $(n_a + n_b - 1) \cdot \mathsf{M}$ (see Bla10 thm 5.8.3)

The polynomial product $A(X) \cdot B(X) \mod M(X)$ can not be computed with less than $(2 \cdot n - t) \cdot \mathsf{M}$ where t is the number of prime factors of M in $\mathbb{F}[X]$.

(see Bla10 thm 5.8.5)

Specifically, cyclic convolution can not be computed with fewer then $(2 \cdot n - t) \cdot \mathsf{M}$ where t is the number of prime factors of $X^n - 1$ in $\mathbb{F}[X]$.

References

- JS20 Ju, Solomonik (2020). Derivation and Analysis of Fast Bilinear Algorithms for Convolution. Github
- Bod07 Bodrato (2007). Towards Optimal Toom-Cook Multiplication for Univariate and Multivariate Polynomials in Characteristic 2 and 0.
- Bla10 Blahut (2010). Fast Algorithms for Signal Processing.
- TAL97 Tolimieri, An, Lu (1997). Algorithms for Discrete Fourier Transform and Convolution.

Appendix: Solving the Chinese Remainder Theorem

Given a set of coprime moduli m_i and values x_i such that $x_i = x \mod m_i$ for some x, we want to reconstruct $x \mod m$ where $m = \prod_i m_i$. To do this we will construct a basis e_i similar to Lagrange basis such that

$$x = \sum_i x_i \cdot e_i \mod m$$

where $e_i \mod m_j$ equals 0 when $i \neq 1$ and 1 when $i = j$. The former implies $e_i = r_i \cdot \prod_{i \neq j} m_j$ for some r_j. The latter implies

$$r_i = \left(\prod_{i \neq j} m_j\right)^{-1} \mod m_i$$

There are multiple solutions $r_i' = r_i + k \cdot m_i$ and this reflects multiple solutions $x' = x + k \cdot m$. By convention the smallest is used. We can write the CRT basis concisely using $[-]_m$ for modular reduction as

$$e_i = \frac{m}{m_i} \cdot \left[\frac{m_i}{m}\right]_{m_i}$$

If the moduli are not coprime we can correct with $m = \operatorname{lcm}(m_i)$. (**Question** does the above solution using $\frac{m}{m_i}$ still hold?) Note that in this case solutions may not exist. For example if m_0 and m_1 have a common factor d and $x_0 \neq x_1 \mod d$ there is no solution. (**Question** what does the above solution produce in this case?).

The CRT establishes a ring isomorphism

$$\mathbb{F}/m \cong \mathbb{F}/m_0 \times \mathbb{F}/m_1 \times \cdots$$

with the maps $x \mapsto ([x]_{m_0}, [x]_{m_1}, \ldots)$ and $(x_0, x_1, \ldots) \mapsto x_0 \cdot e_0 + x_1 \cdot e_1 + \cdots$.

Question. How does this generalize to include derivatives?

Appendix: applications to proof systems

Note the similarity between bilinear algorithms and an R1CS predicate $\varphi(w) \Leftrightarrow (A \cdot w) \circ (B \cdot w) = C \cdot w$. If C is invertible this can be restated as a bilinear algorithm $\beta(a, b) = C^{-1} \cdot ((A \cdot a) \circ (B \cdot b))$ and we have $\varphi(w) \Leftrightarrow \beta(w, w) = w$.

Applying this to the matrix exchange theorem we get that a given R1CS (A, B, C) is equivalent to $\left(A, \left(\mathrm{E} \cdot C^{-1}\right)^\mathsf{T}, \left(\left(B \cdot \mathrm{E}\right)^\mathsf{T}\right)^{-1}\right)$.

Since linear convolutions is equivalent to polynomial multiplication, a polynomial commitment scheme allows for an efficient proof of convolution. To proof $c = a * b$, proof $C(\tau) = A(\tau) \cdot B(\tau)$ at a random point $\tau \in \mathbb{F}$. To proof a size n (nega)cyclic convolution test that the following holds for n term polynomials A, B, C, Q:

$$C(\tau) + Q(\tau) \cdot (\tau^n \pm 1) = A(\tau) \cdot B(\tau)$$

If we have a proof for reduction from FFT to convolution this could be used to proof FFTs.

Appendix: bilinear maps

Given vectors spaces U, V, W over a base field $\mathbb{F}$, a bilinear map is a function $f : U \times V \to W$ such that

$$\begin{aligned} f(u_1 + u_2, v) &= f(u_1, v) + f(u_2, v) \quad & f(\lambda \cdot u, v) &= \lambda \cdot f(u, v) \\ f(u, v_1 + v_2) &= f(u, v_1) + f(u, v_2) \quad & f(u, \lambda \cdot v) &= \lambda \cdot f(u, v) \end{aligned}$$

Bilinear algorithms are bilinear maps, so convolutions and polynomial modular multiplication are bilinear maps. Other examples of bilinear maps are matrix multiplications and the cross product. When $W = \mathbb{F}$, it is also called a bilinear form with further examples such as inner products.

Given bases a bilinear map can be written as a tensor u, v, w.

$$f_k(u, v) = \sum f_{ijk} \cdot u_i \cdot v_j$$

From this a bilinear algorithm follows. For each non-zero element of f_{ijk} create rows in A and B with all zeros except for a single 1 in columns i and j respectively. To C we add a column with the value f_{ijk} in row k. This has complexity $m \cdot \mathsf{M}^\mathsf{c} + n \cdot \mathsf{M}$ where n is the number of nonzero entries and m the number of nontrivial entries.

Appendix: Polynomial composition

Given two polynomials $P(X) = \sum_i p_i \cdot X^i$ and $Q(X) = \sum_i q_i \cdot X^i$, define the composition $R(X) = P(Q(X))$ where the coefficients of R are given by:

$$\begin{aligned} R(X) &= P(Q(X)) \\ &= \sum_i p_i \cdot \left(\sum_j q_j \cdot X^j\right)^i \end{aligned}$$

Multinomial theorem.

$$P(X)^i = \left(\sum_j p_j \cdot X^j\right)^i = \sum_j \frac{i!}{j_1! j_2! j_3! \dots} \cdot p_{j_1} \cdot p_{j_2} \cdot \dots \cdot X^{j_1 + j_2 + \dots}$$

$$P(X)^i = \left(\sum_j q_j \cdot X^j\right)^i = \sum_j \frac{i!}{j_1! j_2! j_3! \dots} \cdot$$

To do

https://arxiv.org/pdf/2201.10369.pdf

https://cr.yp.to/lineartime/multapps-20080515.pdf

https://eprint.iacr.org/2016/504.pdf page 5 describes how to turn convolution theorem into negacyclic.

Fast modular composition. p. 338 in Modern Computer Algebra.

https://relate.cs.illinois.edu/course/cs598evs-f20/
https://relate.cs.illinois.edu/course/cs598evs-f20/f/lectures/03-lecture.pdf

https://arxiv.org/abs/1707.04618

https://www.nature.com/articles/s41586-022-05172-4

https://doc.sagemath.org/html/en/reference/polynomial_rings/sage/rings/polynomial/convolution.html
https://jucaleb4.github.io/files/conv_ba.pdf

https://jucaleb4.github.io/files/poster1.pdf